Corporación Unificada Nacional de Educación Superior
VIGILADA MINEDUCACIÓN

Información general

Fecha de creación	29 – 09 - 2025	**Dirección de operaciones**

Escuela	Ingenierías
Programa	Ingeniería de sistemas.

Identificadores del curso

Código de asignatura	INGS0806
Asignatura	Programación Orientada a Objetos 2
Temática	Persistencia de Datos en Aplicaciones Orientadas a Objetos

Introducción

Cuando desarrollamos una aplicación, ya sea un sistema de inventario, una red social o un videojuego, creamos objetos que viven y operan en la memoria RAM del computador. Estos objetos guardan información crucial: los datos de un cliente, los productos en un carrito de compras, o el puntaje de un jugador. Sin embargo, esta memoria es volátil. En el momento en que la aplicación se cierra o el sistema se apaga, toda esa información se desvanece como si nunca hubiera existido. Aquí es donde surge una pregunta fundamental para cualquier desarrollador de software: ¿cómo podemos hacer que la información de nuestros objetos perdure en el tiempo? La respuesta a esta pregunta es la **persistencia de datos**.

La persistencia de datos es el conjunto de principios y técnicas que permiten que el estado de un objeto sobreviva al proceso que lo creó. En términos simples, es el mecanismo para guardar la información de la memoria (que es temporal) en un medio de almacenamiento permanente, como un archivo en el disco duro o una base de datos. Esta capacidad es la que transforma un programa simple en una aplicación robusta y funcional. Sin persistencia, no existirían los perfiles de usuario, el guardado de progreso en los juegos, las transacciones bancarias en línea o el comercio electrónico. Cada vez que abriéramos una aplicación, empezaríamos desde cero.

Dominar la persistencia de datos es, por tanto, una habilidad no negociable en el desarrollo de software moderno. Nos permite construir sistemas que recuerdan, que aprenden del pasado y que ofrecen una experiencia continua y fiable al usuario. Este documento está diseñado para guiarte a través de los conceptos, herramientas y buenas prácticas para implementar soluciones de persistencia en tus aplicaciones orientadas a objetos. No se trata solo de guardar datos, sino de hacerlo de manera eficiente, segura y escalable. A lo largo de estos capítulos, descubrirás cómo darle memoria a tus aplicaciones, transformando programas efímeros en soluciones de software robustas y permanentes.

Objetivos

El propósito de este módulo es que, al finalizar, puedas diseñar e implementar soluciones de persistencia de datos en aplicaciones orientadas a objetos, seleccionando la estrategia más adecuada (archivos, bases de datos relacionales o no relacionales) según los requerimientos del problema. Este conocimiento es la base para construir aplicaciones completas y profesionales, capaces de gestionar información de manera efectiva a lo largo del tiempo.

Objetivos Específicos

Al completar el estudio de este documento, estarás en capacidad de:

- **Comprender** el "desajuste de impedancia objeto-relacional", identificando por qué es un desafío central al conectar la lógica de la programación orientada a objetos con las bases de datos tradicionales.
- **Implementar** las operaciones básicas de persistencia, conocidas como CRUD (Crear, Leer, Actualizar y Borrar), utilizando un framework de Mapeo Objeto-Relacional (ORM) en un proyecto Java.
- **Utilizar** formatos de intercambio de datos estándar como JSON y XML para la serialización y persistencia de objetos en archivos.
- **Modelar** diagramas de clases que representen entidades del mundo real y que puedan ser mapeados de manera coherente a un esquema de base de datos relacional.

Fundamentos teóricos

Para construir aplicaciones de software complejas y útiles, no basta con darles un comportamiento inteligente; también debemos dotarlas de memoria. Este capítulo explora los conceptos fundamentales que nos permiten almacenar y recuperar la información de nuestros objetos, asegurando que los datos sobrevivan más allá de una simple sesión de uso. Analizaremos desde las estrategias más clásicas hasta los desafíos que dieron origen a las herramientas modernas que usamos hoy en día.

3.1. ¿Qué es la Persistencia de Datos?

En la programación orientada a objetos (POO), todo gira en torno a los objetos. Un objeto es una entidad que tiene un **estado** (sus atributos o datos) y un **comportamiento** (sus métodos o las acciones que puede realizar). Por ejemplo, un objeto Estudiante tiene un estado (nombre: "Ana", código: "12345") y un comportamiento (matricularMateria(), calcularPromedio()).

3.2. Estrategias Clásicas de Persistencia

Dos de las estrategias más comunes son la serialización y el uso de archivos planos. La **serialización** convierte el estado de un objeto en una secuencia de bytes que puede ser almacenada. Por otro lado, los **archivos de texto plano (CSV)** o los **lenguajes de marcas (XML)** permiten estructurar la información de manera legible (Toharia Rabasco & Raya González, 2015).

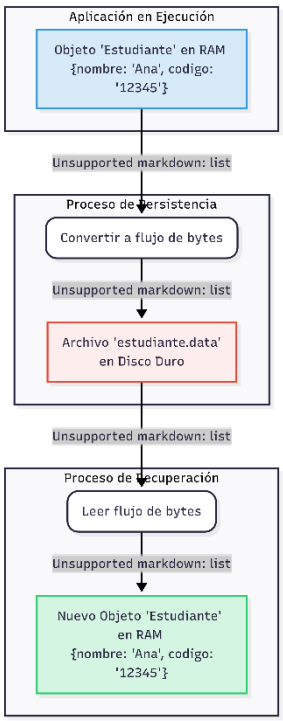

Figura 1 *Diagrama de Flujo del Proceso de Serialización y Deserialización*
Fragmento de código

Diagrama que muestra un objeto en RAM siendo serializado a un archivo en disco, y luego leído y deserializado de vuelta a un objeto en RAM.

3.3. El Mundo de las Bases de Datos

Cuando las aplicaciones necesitan gestionar grandes volúmenes de datos, entran en juego las bases de datos, sistemas especializados en almacenar y consultar datos de forma eficiente (Kroenke, 2003). Se dividen en dos grandes familias:

- **Bases de Datos Relacionales (SQL):** Organizan los datos en tablas, filas y columnas, utilizando el Lenguaje de Consulta Estructurado (SQL) para su manipulación (Martín Escofet, 2002).
- **Bases de Datos No Relacionales (NoSQL):** Ofrecen modelos de datos más flexibles, ideales para datos que no encajan bien en una estructura de tabla rígida.

3.4. El Desafío Central: Desajuste de Impedancia Objeto-Relacional

El problema fundamental al conectar la POO con las bases de datos relacionales es la diferencia entre el paradigma de objetos (con herencia, polimorfismo) y el paradigma relacional (plano y tabular). Este conflicto se conoce como el **desajuste de impedancia objeto-relacional**. La investigación en esta área ha buscado proponer modelos formales que faciliten esta "traducción" (Hernández González, 2009).

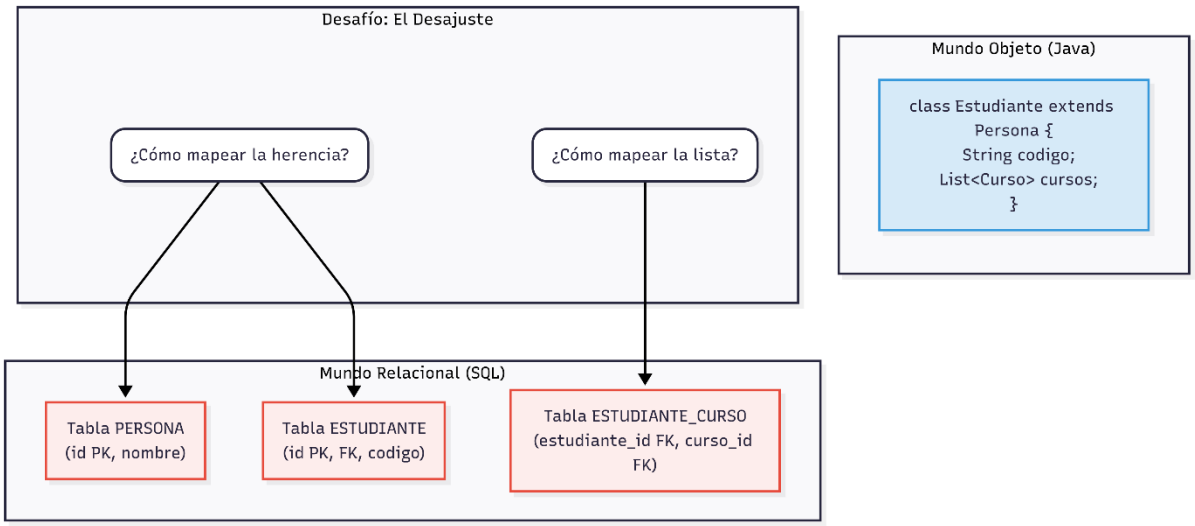

Figura 2 *Mapa Conceptual del Desajuste de Impedancia Objeto-Relacional*

Mapa conceptual que compara una estructura de clases con herencia y listas en Java con su compleja representación en múltiples tablas SQL.

3.5. Introducción a los ORM (Object-Relational Mapping)

Para resolver el problema del desajuste de impedancia, surgieron herramientas llamadas **ORM (Mapeo Objeto-Relacional)**. Un ORM es un software que actúa como un "traductor" automático entre el paradigma orientado a objetos de tu aplicación y el paradigma relacional de la base de datos. En lugar de escribir sentencias SQL a mano (Camuña Rodríguez, 2025), trabajas directamente con tus objetos, y el ORM se encarga de generar el código necesario para las operaciones CRUD (Crear, Leer, Aktualizar, Borrar).

Aplicaciones en la vida cotidiana

La teoría de la persistencia de datos solo cobra sentido cuando la vemos en acción. Los conceptos como serialización, bases de datos SQL y NoSQL no son ideas abstractas, sino herramientas que resuelven problemas cotidianos para millones de personas. En este capítulo, exploraremos tres escenarios prácticos que ilustran cómo diferentes estrategias de persistencia se aplican en el software que usamos a diario. Desde guardar una simple configuración hasta gestionar complejas transacciones comerciales, veremos cómo la persistencia es el motor silencioso que hace que la tecnología funcione para nosotros.

2.1. Persistencia con Archivos: El Caso de la Configuración de una App

Imagina que descargas una aplicación de escritorio en tu computador, como un IDE de programación (NetBeans) o un videojuego. La primera vez que la abres, eliges el idioma en español, seleccionas el "modo oscuro" y ajustas el tamaño de la letra. Cierras la aplicación y, al día siguiente, la vuelves a abrir. Mágicamente, todas tus preferencias siguen ahí. ¿Cómo es posible?

- **El Problema:** La aplicación necesita "recordar" las elecciones del usuario entre sesiones. Estos datos de configuración son únicos para cada usuario pero no son complejos ni requieren ser consultados por miles de personas a la vez.
- **La Solución Cotidiana:** La estrategia más eficiente para este caso es la persistencia en archivos. La aplicación crea un objeto, por ejemplo ConfiguracionUsuario, que contiene atributos como idioma="es", tema="oscuro", tamanoFuente=14. Cuando cierras la aplicación, esta serializa el objeto ConfiguracionUsuario y lo guarda en un archivo local en tu computador, comúnmente en formato **JSON** (JavaScript Object Notation) o **XML** (eXtensible Markup Language). Estos lenguajes de marcas son fundamentales para los sistemas de gestión de información modernos (Toharia Rabasco & Raya González, 2015).

- **Análisis de Formatos:**
    - **JSON:** Es preferido por su simplicidad y legibilidad. Es nativo para JavaScript, lo que lo hace ideal para aplicaciones web, pero su simplicidad lo ha convertido en un estándar para archivos de configuración en muchos entornos.

JSON
{ "idioma": "es-CO", "tema": "oscuro", "tamanoFuente": 14 }

    - **XML:** Es más verboso, pero también más potente para describir estructuras de datos complejas y metadatos, gracias a su sistema de

etiquetas y atributos. La validación de la estructura de un archivo XML contra un esquema (XSD) es una de sus grandes fortalezas en entornos empresariales (Kim-Park, 2011).

XML
```
<configuracionUsuario>
   <idioma>es-CO</idioma>
   <tema>oscuro</tema>
   <tamanoFuente>14</tamanoFuente>
</configuracionUsuario>
```

- **¿Por qué funciona tan bien?** Es una solución ligera, rápida y no requiere instalar un sistema de base de datos completo. Al iniciar, la aplicación simplemente lee este archivo, lo deserializa y reconstruye el objeto de configuración en memoria. Es el enfoque perfecto para datos de bajo volumen y específicos del entorno local.

2.2. El Carrito de Compras de un E-commerce Colombiano

Pensemos en una tienda online local, como "Cafés de mi Tierra", que vende café especial de diferentes regiones de Colombia.

- **El Problema:** El estado del carrito de compras debe persistir a través de diferentes sesiones y dispositivos. Además, el sistema debe controlar el inventario de forma concurrente para asegurar que el café que un cliente quiere comprar esté disponible.
- **La Solución con Base de Datos Relacional (SQL):** Un archivo JSON no es suficiente aquí. Necesitamos un sistema transaccional robusto que garantice la integridad de los datos. La solución ideal es una **base de datos relacional** (como PostgreSQL o SQL Server). El **diseño de la base de datos** es el pilar de la aplicación (Capacho & Nieto Bernal, 2017).
- **Interacciones y Transacciones:**
    1. **Añadir al Carrito:** Cuando un cliente añade un producto, se ejecuta una sentencia INSERT en una tabla CARRITO_ITEMS, asociando el id_usuario, id_producto y la cantidad.
    2. **Ver Carrito:** Se ejecuta una consulta SELECT con un JOIN entre las tablas CARRITO_ITEMS y PRODUCTOS para mostrar al usuario la descripción y el precio de los ítems que ha seleccionado.
    3. **Finalizar Compra (La Transacción Crítica):** Este es el momento más importante. El sistema debe ejecutar varias operaciones como un todo indivisible. Se inicia una **transacción** que:
        - Verifica el stock del producto en la tabla PRODUCTOS (SELECT ... FOR UPDATE para bloquear la fila y evitar que otro la compre al mismo tiempo).

- Si hay stock, crea un registro en la tabla PEDIDOS.
- Inserta los detalles del carrito en la tabla DETALLES_PEDIDO.
- Actualiza el inventario en la tabla PRODUCTOS (UPDATE PRODUCTOS SET stock = stock - cantidad ...).
- Elimina los ítems del CARRITO_ITEMS. Si cualquiera de estos pasos falla, la transacción completa se revierte (*rollback*), y la base de datos vuelve al estado anterior, evitando inconsistencias como vender un producto sin stock. El **lenguaje SQL** es la herramienta que permite estas operaciones complejas (Martín Escofet, 2002).

2.3. Gestión de Pacientes en un Centro de Salud en Bogotá

Una enfermera jefe en un centro de salud necesita acceder rápidamente al historial de un paciente.

- **El Problema:** Los datos son heterogéneos y no tienen una estructura fija. Un paciente puede tener una radiografía, otro puede tener 20 resultados de laboratorio, y un tercero puede tener notas de múltiples especialistas. Forzar esto en una estructura relacional rígida llevaría a tener tablas con cientos de columnas, la mayoría vacías para casi todos los pacientes, lo cual es ineficiente y difícil de mantener.
- **La Solución con Base de Datos No Relacional (NoSQL):** Este es un escenario perfecto para una **base de datos documental** como MongoDB. Cada paciente se almacena como un "documento" (en formato BSON, similar a JSON) que contiene toda su información.
- **Ventajas de la Flexibilidad:**
  - **Esquema Dinámico:** Si mañana se introduce un nuevo tipo de prueba genética, no es necesario alterar el diseño de la base de datos (ALTER TABLE). Simplemente, los nuevos pacientes tendrán un nuevo campo en su documento.
  - **Rendimiento en Lectura:** Toda la información de un paciente se puede recuperar en una sola operación de lectura, sin necesidad de realizar complejos JOINs entre 10 o 15 tablas diferentes, lo cual es crucial en una situación de urgencia.
  - **Escalabilidad Horizontal:** A medida que el hospital crece, se pueden añadir más servidores para distribuir la carga, una característica nativa de muchas bases de datos NoSQL.

Este enfoque flexible se alinea mucho mejor con la naturaleza semi-estructurada de los datos del mundo real en muchos dominios.

Casos de estudio

Aquí analizamos cómo se resuelven los problemas de persistencia de manera sistemática y escalable en un proyecto de software empresarial.

3.1. Caso de Estudio: Sistema de Gestión y Reservas para una Flota de Transporte Intermunicipal

- **“**Transportes Andinos S.A.S." enfrentaba pérdidas económicas directas por su sistema manual. La sobreventa de tiquetes resultaba en compensaciones a clientes y daño a la reputación. La falta de datos centralizados impedía optimizar las rutas más rentables. La asignación de conductores y buses era un rompecabezas logístico que consumía horas de trabajo administrativo y era propenso a errores humanos costosos (ej. asignar un bus en mantenimiento a una ruta larga).

- **Solución Aplicada: Una Arquitectura por Capas** El equipo de desarrollo optó por una arquitectura de 3 capas, un estándar en la industria para organizar la complejidad:
    1. **Capa de Presentación:** Se desarrollaron dos interfaces: una aplicación web para que los clientes consulten rutas y compren tiquetes, y una aplicación de escritorio para los agentes de ventas en las terminales.
    2. **Capa de Lógica de Negocio:** Aquí reside el "cerebro" del sistema. Se crearon servicios como ReservaService y RutaService. Por ejemplo, ReservaService contenía el método crearReserva(datosReserva), que orquestaba todas las reglas de negocio: verificar la disponibilidad, validar el pago y confirmar la reserva.
    3. **Capa de Acceso a Datos (Persistencia):** Esta capa es la única que se comunica directamente con la base de datos. Se implementó usando el **patrón DAO (Data Access Object)**. Se crearon interfaces como IReservaDAO y sus implementaciones con Hibernate, como ReservaDAOImpl. Esto desacopla la lógica de negocio de la tecnología de persistencia. Mañana, si se decidiera cambiar de Hibernate a otra tecnología, solo habría que cambiar la implementación del DAO, sin tocar la capa de negocio. Este diseño se basa en **modelos de persistencia formales** que separan el qué del cómo (Hernández González, 2009).

Diseño de la Base de Datos y Mapeo: El equipo de bases de datos, siguiendo principios de **diseño de base de datos** (Capacho & Nieto Bernal, 2017), creó un esquema relacional normalizado en SQL Server (Wanumen Silva, 2018). Luego, los desarrolladores de la capa de persistencia usaron las anotaciones de JPA/Hibernate para mapear cada clase del dominio (Figura 3) a las tablas correspondientes.

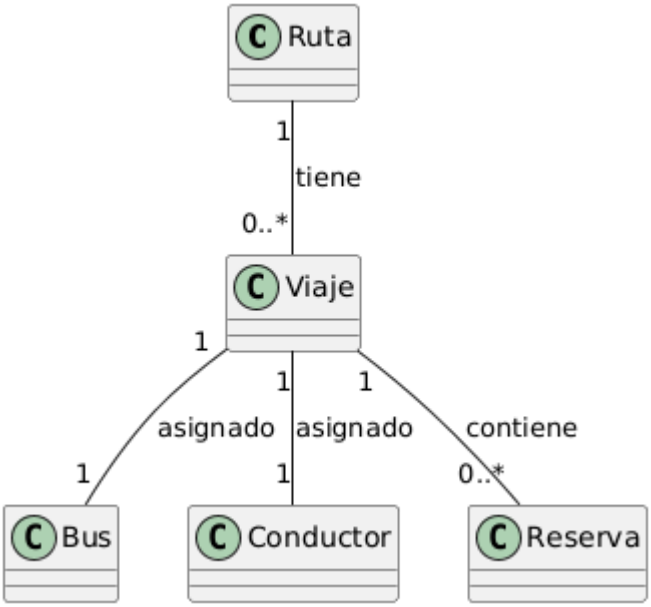

Figura 3 *Diagrama de Clases UML Simplificado del Sistema de Reservas*

Diagrama UML con las clases Ruta, Viaje, Bus, Conductor y Reserva, mostrando sus relaciones.

- **Desafíos Durante la Implementación:**
  - **Migración de Datos:** El mayor reto fue migrar los datos históricos desde hojas de cálculo a la nueva base de datos relacional. Se tuvieron que crear scripts para limpiar, transformar y cargar miles de registros, asegurando su consistencia.
  - **Gestión del Cambio:** La resistencia del personal acostumbrado al sistema manual fue un desafío. Se requirió un plan de capacitación intensivo para asegurar la adopción de la nueva herramienta.

- **Impacto Medible y Cualitativo:**
  - **Cuantitativo:** Además de la reducción del 95% en sobreventas, el sistema permitió un **análisis de rentabilidad por ruta**, lo que llevó a la optimización de horarios, resultando en un aumento del 15% en la ocupación promedio de los buses. La automatización de la asignación de vehículos redujo los costos administrativos en un 40%.
  - **Cualitativo:** La percepción de la marca mejoró notablemente. Los clientes ahora veían a "Transportes Andinos" como una empresa moderna y fiable. Internamente, la moral de los empleados aumentó al eliminar tareas manuales tediosas y propensas a conflictos.

Herramientas y técnicas

Este capítulo es una guía práctica para comenzar a trabajar con persistencia en Java.

4.1. Mapeo Objeto-Relacional con Hibernate (ORM)

Hibernate es el puente entre nuestro código Java y la base de datos relacional.

- **Configuración y SessionFactory:** La configuración en hibernate.cfg.xml es leída al inicio de la aplicación para crear un objeto SessionFactory. Este objeto es pesado y costoso de crear, por lo que se crea una sola vez y se reutiliza durante toda la vida de la aplicación.

- **EntityManager y la Sesión:** De la SessionFactory obtenemos un EntityManager (o Session), que representa una conversación con la base de datos. Es un objeto ligero que se crea y se destruye para cada unidad de trabajo.

- **Ejercicio Práctico Guiado (CRUD):**
    1. **La Entidad Estudiante:**

Java
```
@Entity @Table(name = "estudiantes")
public class Estudiante {
   @Id @Column(name = "codigo_estudiante")
   private String codigo;
   @Column(name = "nombre_completo")
   private String nombre;
   private String carrera;
   // Getters, Setters, Constructores...
}
```

    2. **El Código de Persistencia:**

Java
```
// Suponiendo que tenemos una clase HibernateUtil para gestionar la
SessionFactory
EntityManager em = HibernateUtil.getEntityManager();

// CREATE
em.getTransaction().begin();
```

```
Estudiante nuevo = new Estudiante("202401", "Carlos Moncada", "Ingeniería de Software");
em.persist(nuevo);
em.getTransaction().commit();

// READ
Estudiante encontrado = em.find(Estudiante.class, "202401");
System.out.println("Leído: " + encontrado.getNombre());

// UPDATE
em.getTransaction().begin();
encontrado.setCarrera("Ingeniería de Sistemas");
em.merge(encontrado);
em.getTransaction().commit();

// DELETE
em.getTransaction().begin();
em.remove(encontrado);
em.getTransaction().commit();

em.close();
```

- Esta aproximación al **procesamiento de bases de datos** a través de un ORM es un pilar de la ingeniería de software moderna (Kroenke, 2003).

4.2. Consultas con HQL (Hibernate Query Language)

HQL es más potente que un simple find. Permite expresar consultas complejas de manera orientada a objetos.

- **HQL vs. SQL:**
  - SQL: SELECT nombre_completo FROM estudiantes WHERE carrera = 'Ingeniería de Sistemas';
  - HQL: SELECT e.nombre FROM Estudiante e WHERE e.carrera = :carrera La sintaxis es similar, pero HQL usa nombres de clases y propiedades, no de tablas y columnas.

- **Ejemplos de Consultas Avanzadas:**
  - **Consulta con JOIN:** Para obtener todas las reservas de un viaje específico.

Fragmento de código

```
SELECT r FROM Reserva r JOIN r.viaje v WHERE v.id = :viajeId
```

- **Consulta con Agregación:** Para contar cuántos estudiantes hay en cada carrera.

Fragmento de código
```
SELECT e.carrera, COUNT(e) FROM Estudiante e GROUP BY e.carrera
```

- Estas consultas demuestran el poder de los **lenguajes de definición y modificación de datos** adaptados al paradigma de objetos (Camuña Rodríguez, 2025).

4.3. Trabajo con Archivos JSON usando la librería Gson

La librería Gson de Google es una herramienta excelente para la serialización.

- **Serializando una Colección:** Un caso de uso muy común es guardar una lista de objetos.

Java
```
List<Producto> inventario = new ArrayList<>();
inventario.add(new Producto("CAF-001", "Café Nariño", 35000.0));
inventario.add(new Producto("PAN-002", "Mogolla Chicharrona", 5000.0));

Gson gson = new Gson();
String jsonInventario = gson.toJson(inventario);
// El resultado es un string que representa un array de objetos JSON
// [{"id":"CAF-001",...},{"id":"PAN-002",...}]
```

- **Deserializando una Colección:** Para convertir el string de vuelta a una lista de objetos, necesitamos usar un TypeToken para que Gson entienda la estructura genérica List<Producto>.

Java
```
Type tipoLista = new TypeToken<ArrayList<Producto>>(){}.getType();
List<Producto> inventarioRecuperado = gson.fromJson(jsonInventario, tipoLista);
```

Conclusión

Hemos completado un viaje exhaustivo a través del mundo de la persistencia de datos. Comenzamos con un problema fundamental y aparentemente simple: cómo evitar que la información de nuestras aplicaciones se pierda al cerrarlas. Esta pregunta nos llevó a explorar los cimientos teóricos, desde la serialización de objetos hasta la compleja pero poderosa arquitectura de las bases de datos. Vimos cómo el **desajuste de impedancia objeto-relacional** no es un problema menor, sino un desafío central en la ingeniería de software que ha impulsado la creación de herramientas sofisticadas como los ORM.

Luego, pusimos estos conceptos en el contexto del mundo real. Analizamos cómo un simple archivo **JSON** resuelve elegantemente la necesidad de guardar la configuración de una aplicación, cómo la estructura y transaccionalidad de una **base de datos SQL** son indispensables para la fiabilidad de un e-commerce, y cómo la flexibilidad de **NoSQL** se adapta perfectamente a la data heterogénea del sector salud. Profundizamos en un caso de estudio profesional, donde vimos que la persistencia no es solo una capa de código, sino una decisión de **arquitectura** con un impacto directo y medible en el éxito de un negocio. Finalmente, nos equipamos con las herramientas prácticas, aprendiendo a implementar operaciones **CRUD con Hibernate** y a manejar la serialización de datos con **Gson**.

La persistencia de datos es, en esencia, el arte de darle memoria al software. Es una habilidad que trasciende un lenguaje o un framework específico. Comprender los principios de cómo estructurar, guardar y recuperar datos de manera eficiente y segura te permitirá construir aplicaciones más robustas, escalables y valiosas. El camino del desarrollador de software es un aprendizaje continuo. Te invito a que veas los conceptos de este documento no como un punto final, sino como un sólido punto de partida. Experimenta, construye tus propios proyectos, enfrenta los desafíos de la gestión de datos y sigue aprendiendo, pues en el mundo digital, los datos son el activo más importante.

Preguntas para la Reflexión:

1. Imagina que debes diseñar la base de una red social para una comunidad de artistas en Colombia. Los usuarios publican "obras" que pueden ser imágenes, videos cortos o textos. Los usuarios pueden seguirse entre sí y comentar en las obras. ¿Qué estrategia de persistencia elegirías? ¿Una base de datos relacional, una documental (NoSQL) o quizás una base de datos de grafos? Justifica tu decisión pensando en las consultas principales que el sistema deberá realizar (ej. "mostrar las obras de los artistas que sigo").

2. La persistencia de datos implica guardar información sobre los usuarios. En el caso del sistema de salud que discutimos, se guarda información médica sensible. ¿Qué responsabilidades éticas y legales (como la Ley de Habeas Data en

Colombia) crees que un desarrollador de software debe considerar al diseñar la capa de persistencia de una aplicación que maneja datos personales?

Referencias

Camuña Rodríguez, J. F. (2025). *Lenguajes de definición y modificación de datos SQL. IFCT0310* (1.ª ed.). IC Editorial.

Capacho, J. R., & Nieto Bernal, W. (2017). *Diseño de base de datos*. Universidad del Norte.

Hernández González, A. (2009). Un modelo de persistencia para el diseño de la base de datos basado en un modelo de objetos formal. *Revista Investigación Operacional, 23*(3). Editorial Universitaria.

Kim-Park, D. S. (2011). Aplicación de un oráculo de prueba automatizado a la evaluación de salidas de programas basados en XML. *Revista Española de Innovación, Calidad e Ingeniería del Software, 7*(1), 6-21. Asociación de Técnicos de Informática (ATI).

Kroenke, D. M. (2003). *Procesamiento de bases de datos: fundamentos, diseño e implementación*. Pearson Educación.

Martín Escofet, C. (2002). El lenguaje SQL. *UOC Papers*.

Medina Serrano, S. (2015). *SQL Server 2014: soluciones prácticas de administración*. RA-MA Editorial.

Toharia Rabasco, J. P. & Raya González, L. (2015). *Lenguajes de marcas y sistemas de gestión de la información*. RA-MA Editorial.

Wanumen Silva, L. F. (2018). *Bases de datos en SQL server*. Ecoe Ediciones.